

Solving Differential Equations with Neural Networks

Tatsuro Senga

First-order Ordinary Differential Equation

First-order Ordinary Differential Equation

Consider the following ODE:

$$\frac{d\Psi(x)}{dx} = e^{-2x} - 2\Psi(x)$$

Initial condition: $\Psi(0) = 0.1$

Domain: $x \in [0, 2]$

Solution Approaches:

- Classical analytical method (integrating factor)
- Neural network approximation (our focus)

Given equation:
$$\frac{d\Psi(x)}{dx} = e^{-2x} - 2\Psi(x)$$

Initial condition:
$$\Psi(0) = 0.1, \quad 0 \leq x \leq 2$$

Solution Steps:

- 1 Rewrite as standard form: $\frac{d\Psi}{dx} + 2\Psi = e^{-2x}$
- 2 Find integrating factor: $\mu(x) = e^{\int 2dx} = e^{2x}$
- 3 Multiply both sides: $e^{2x} \frac{d\Psi}{dx} + 2e^{2x}\Psi = 1$
- 4 Recognize: $\frac{d}{dx}(e^{2x}\Psi) = 1$
- 5 Integrate: $e^{2x}\Psi = x + C$
- 6 Solve for Ψ : $\Psi = e^{-2x}(x + C)$
- 7 Apply initial condition: $\Psi = e^{-2x}(x + 0.1)$

Neural Network Approach: Overview

Key Components:

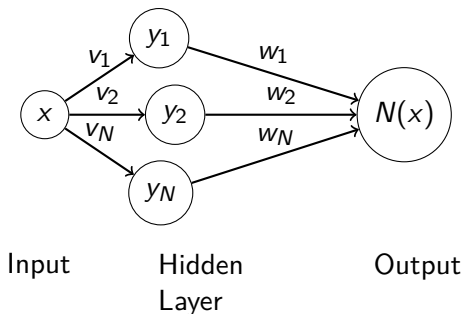
- Trial Solution: $\Psi(x, p) = A + xN(x, p)$
- Neural Network $N(x, p)$ with parameters p
- Automatic satisfaction of initial condition: $\Psi_t(0, p) = A$

Network Architecture:

- Input layer: 1 neuron (x)
- Hidden layer: N neurons
- Output layer: 1 neuron
- Activation function: Sigmoid $f(x) = \frac{1}{1+e^{-x}}$

Neural Network Architecture

- Single hidden layer with N neurons
- Input: $x_i \in [0, 2]$
- Output: Approximation of $\Psi(x_i)$
- Trial solution form: $\Psi(x_i) = A + x_i N(x_i)$



Hidden Layer Processing:

$$y_i = f(v_i x + \theta_i)$$

Output Computation:

$$N(x, p) = \sum_{i=1}^N w_i y_i$$

Parameter Vector:

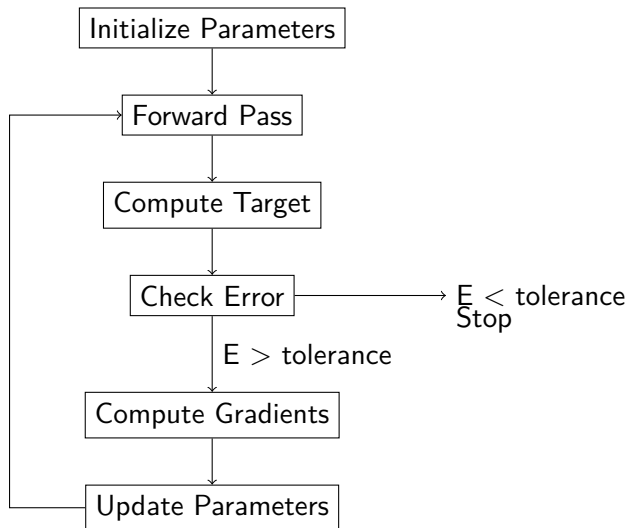
$$p = (v_1, \dots, v_N, \theta_1, \dots, \theta_N, w_1, \dots, w_N)$$

ODE: $\frac{d\Psi(x)}{dx} = f(x, \Psi(x))$

Key Steps:

- 1 Discretize domain: $x_i \in [0, 1]$ for $i = 1, \dots, n$.
- 2 Initialize parameters
- 3 Forward pass: Compute $\Psi(x, p)$ and its derivative $\frac{d\Psi(x)}{dx}$
- 4 Compute $E(p) = \sum_{i=1}^n \left\{ \frac{d\Psi(x_i)}{dx_i} - f(x_i, \Psi(x_i)) \right\}^2$
- 5 Update parameters using quasi-Newton method
- 6 Repeat until convergence ($E(p)$ being sufficiently small)

Training Loop: Implementation



Initialization:

- Input weights: $v \in \mathbb{R}^{5 \times 1}$
- Hidden biases: $\theta \in \mathbb{R}^{5 \times 1}$
- Output weights: $w \in \mathbb{R}^{5 \times 1}$

Xavier/Glorot Initialization:

$$v \sim \mathcal{N}(0, \sqrt{2/1})$$

$$\theta \sim \mathcal{N}(0, \sqrt{2/1})$$

$$w \sim \mathcal{N}(0, \sqrt{2/5})$$

Forward Pass Computations

Hidden Layer:

$$z = v \cdot x + \theta$$

$$y = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Output Layer:

$$N(x) = w^T y$$

$$\Psi_t(x) = A + xN(x)$$

Derivative:

$$\frac{d\Psi_t}{dx} = N(x) + x \frac{dN}{dx}$$

Training Loop

1. Forward Propagation:

$$[\Psi, \frac{d\Psi}{dx}] = \text{forward_pass}(x, v, \theta, w, A)$$

2. Target Computation:

$$\text{target} = e^{-2x} - 2\Psi(x)$$

3. Error Calculation:

$$E = \frac{1}{n} \sum_{i=1}^n \left(\frac{d\Psi(x_i)}{dx_i} - \text{target}_i \right)^2$$

4. Gradient Computation:

$$[\nabla_v E, \nabla_\theta E, \nabla_w E] = \text{compute_gradients}(x, v, \theta, w, \text{target}, A)$$

5. Parameter Update:

$$v = v - \alpha \nabla_v E, \theta = \theta - \alpha \nabla_\theta E, w = w - \alpha \nabla_w E$$

Training Loop: Parameter Updates

Gradient Computation:

$$[\nabla_v E, \nabla_\theta E, \nabla_w E] = \text{compute_gradients}(x_{\text{train}}, v, \theta, w, \text{target}, 0.1)$$

Parameter Updates:

$$v = v - \alpha \nabla_v E$$

$$\theta = \theta - \alpha \nabla_\theta E$$

$$w = w - \alpha \nabla_w E$$

where $\alpha = 0.1$ is the learning rate

Quasi-Newton Method: Parameter Updates

General Update Form:

$$p_{k+1} = p_k - \alpha_k B_k^{-1} \nabla E(p_k)$$

where:

- p_k is parameter vector (v, θ, w) at iteration k
- B_k approximates the Hessian matrix
- α_k is the step size
- $\nabla E(p_k)$ is our computed gradient

In Our Implementation:

$$\text{Update: } p_{k+1} = p_k - \alpha \nabla E(p_k)$$

$$\text{where: } \nabla E(p_k) \approx \frac{E(p_k + \epsilon) - E(p_k - \epsilon)}{2\epsilon}$$

Goal: Use finite differences to compute $\frac{\partial E}{\partial v_i}$ for each parameter

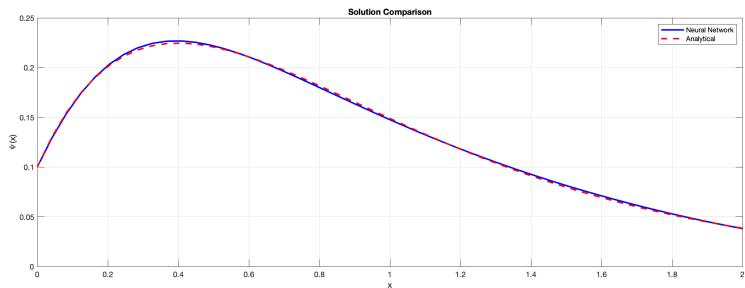
Central Difference Implementation:

$$\begin{aligned}\frac{\partial E}{\partial v_i} &\approx \frac{E(v_i + \epsilon) - E(v_i - \epsilon)}{2\epsilon} \\ &= \frac{1}{2\epsilon} [\text{mean}((\text{dPsi}^+ - \text{target})^2) - \text{mean}((\text{dPsi}^- - \text{target})^2)]\end{aligned}$$

Process:

- 1 Perturb parameter: $v_i^\pm = v_i \pm \epsilon$
- 2 Evaluate perturbed solutions: $\text{dPsi}^\pm = \text{forward_pass}(x, v^\pm, \theta, w, A)$
- 3 Compute error difference and scale by $\frac{1}{2\epsilon}$

Result



Automatic Differentiation (Backpropagation):

- Computes exact gradients
- More efficient than numerical differentiation
- Requires chain rule implementation through network
- Commonly used in deep learning frameworks

Hyperparameter Selection:

- Learning rate ($\alpha = 0.1$)
 - Too high: instability
 - Too low: slow convergence
- Maximum epochs (20,000)
 - Balance between convergence and computation time
- Error tolerance (10^{-6})
 - Trade-off between accuracy and training time

Black-Scholes PDE

Black-Scholes PDE

Consider the Black-Scholes PDE:

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0$$

Terminal condition: $V(S, T) = \max(S - K, 0)$

Domain: $S \in [0, 2], \tau \in [0, 1]$

Parameters:

- Strike price: $K = 1$
- Risk-free rate: $r = 0.05$
- Volatility: $\sigma = 0.3$
- Time to maturity: $T = 1$

Neural Network Approach: Overview

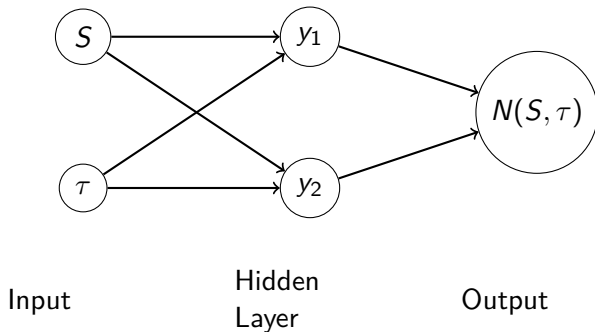
Key Components:

- Trial Solution: $\Psi_t(S, \tau) = \max(S - K, 0) + \tau SN(S, \tau, p)$
- Neural Network $N(S, \tau, p)$ with parameters p
- Automatic satisfaction of terminal condition at $\tau = 0$

Network Architecture:

- Input layer: 2 neurons (S, τ)
- Hidden layer: 2 neurons
- Output layer: 1 neuron
- Activation function: Sigmoid $f(x) = \frac{1}{1+e^{-x}}$

Neural Network Architecture



Initialization:

- Input weights: $v \in \mathbb{R}^{2 \times 2}$
- Hidden biases: $\theta \in \mathbb{R}^2$
- Output weights: $w \in \mathbb{R}^2$

Xavier/Glorot Initialization:

$$v \sim \mathcal{N}(0, \sqrt{2/2})$$

$$\theta \sim \mathcal{N}(0, \sqrt{2/2})$$

$$w \sim \mathcal{N}(0, \sqrt{2/2})$$

Forward Pass Computations

Hidden Layer:

$$z = v_1 S + v_2 \tau + \theta$$

$$y = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Output Layer:

$$N(S, \tau) = w^T y$$

$$\Psi_t(S, \tau) = \max(S - K, 0) + \tau SN(S, \tau)$$

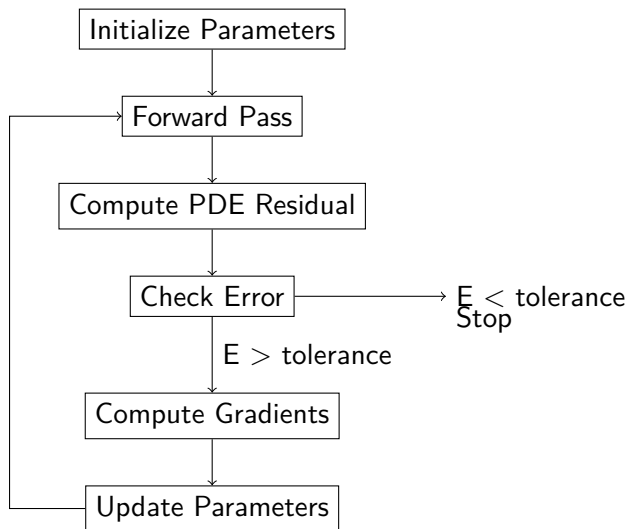
Derivatives:

$$\frac{\partial \Psi_t}{\partial \tau} = SN + \tau S \frac{\partial N}{\partial \tau}$$

$$\frac{\partial \Psi_t}{\partial S} = 1_{S > K} + \tau N + \tau S \frac{\partial N}{\partial S}$$

$$\frac{\partial^2 \Psi_t}{\partial S^2} = \tau \left(2 \frac{\partial N}{\partial S} + S \frac{\partial^2 N}{\partial S^2} \right)$$

Training Loop



PDE Residual:

$$R(S, \tau) = \frac{\partial \Psi_t}{\partial \tau} - rS \frac{\partial \Psi_t}{\partial S} - \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 \Psi_t}{\partial S^2} + r \Psi_t$$

Error Function:

$$E(p) = \frac{1}{n} \sum_{i=1}^n [R(S_i, \tau_i)]^2$$

Parameter Updates:

$$v = v - \alpha \nabla_v E$$

$$\theta = \theta - \alpha \nabla_\theta E$$

$$w = w - \alpha \nabla_w E$$

Grid Setup:

- Non-uniform S grid:
 - Dense near strike price K
 - Sparse in tails
- Uniform τ grid in $[0,1]$

Hyperparameters:

- Learning rate: $\alpha = 0.01$
- Maximum iterations: 10,000
- Error tolerance: 10^{-5}
- Grid points: 50×50

Results Visualization

- Surface plot shows:
 - Payoff profile at $t=0$
 - Time value increasing with
 - Smooth transition around strike price
- Error analysis:
 - PDE residual convergence
 - Comparison with analytical solution

